

bridge_tunnel_nav

A POMDP navigation environment with a bridge / mine / detour strategy choice, built as a substrate for mechanistic interpretability

Abstract

BRIDGE_TUNNEL_NAV is a small partially-observable navigation environment in which an agent crosses open procedural terrain to reach a narrow goal door. The direct route is blocked by obstacles the agent can deal with in three distinct ways: **bridge** a lake (place wood on water), **mine** a mountain (turn rock to grass), or **walk around** it. Impassable tree forests line the top and bottom edges so the agent is funnelled through the obstacle-filled middle, where it must repeatedly choose among the three strategies. The point of the design is *not* the navigation itself but the **internal computation it induces**: under partial observation the agent must infer the shape and type of the obstacle ahead (a *belief*) and select a crossing strategy (an *action mode*). We train two agents on the identical task — a recurrent PPO baseline and a pure-JAX DreamerV3 world model — both reaching 100% success, and use them as a testbed for probing and **steering** those belief and strategy representations in activation space. The concrete target is to make an agent that solves the task reliably but **never digs tunnels**, by suppressing the “mine” strategy through an activation-space intervention rather than an action mask.

1 Idea and motivation

We want a task that is (i) trivial to solve, so competence is not the bottleneck, yet (ii) forces a small number of *discrete, legible decisions* that depend on a *partially-observed latent* (the obstacle ahead). Such a task is an ideal substrate for mechanistic interpretability: there should be a low-dimensional *belief* subspace (is the obstacle water or rock; how wide is it) and a *strategy* subspace (bridge vs. mine vs. detour) in the agent’s recurrent state, and we can test whether those subspaces are (a) linearly decodable and (b) *causally* steerable. The headline experiment is suppression: keep the agent fully competent while removing one option (tunneling) by editing activations — the recurrent analogue of the “cheese vector” steering result on maze-solving policy networks.

Two environments. We study the belief–strategy relationship in two complementary settings. We *first* present a deliberately **controlled** variant, `bridge_tunnel_commit`, in which both the belief and the skill are made *explicit and discrete* — a clean ground-truth map from a labeled belief to an enforced, committed skill (§2). We *then* present the open `bridge_tunnel` task (the remainder of the paper), where neither is labeled nor enforced: the belief is a continuous, partially-observed inference and the strategy is a soft, reversible tendency. The controlled variant is the place to first establish that a belief→skill map exists and is legible; the open task is where the harder probing/steering programme of §9 lives.

2 The bridge_tunnel_commit environment: a controlled belief–skill testbed

Why a controlled variant. In the open task (below) the agent’s belief — the type and size of the obstacle ahead — is a continuous, noisy inference, and its “strategy” is a soft tendency it can revise mid-obstacle (poke a lake, back out, detour). That makes *belief* and *skill* useful descriptions for us, but neither is a clean ground-truth variable to probe against. `bridge_tunnel_commit` pins both down while keeping the terrain, observation format, reward and agents otherwise identical:

- **Labeled belief = map category.** Every map belongs to one of three labeled categories, drawn uniformly: BALANCED (14%/14% water/rock), LAKES (water-dominated, $\approx 80/20$), and ROCKY (rock-dominated, $\approx 20/80$). The dominant obstacle is the latent the agent must infer from its egocentric crop, and we hold the exact category label for every episode (Fig. 1).
- **Enforced skill = implicit, irreversible commitment.** The action space is `Discrete(6)` (four moves, BUILD, MINE). Commitment is *implicit*: the **first successful** BUILD (bridging a faced water cell) locks the agent into building for the rest of the episode; the first successful MINE locks it into mining. Thereafter the opposite tool is permanently disabled — using it is a no-op with a small penalty. So the agent commits simply by *choosing to cross its first obstacle*; there is no separate commit action it could spam blind at the spawn, so the commitment naturally tracks the first obstacle it decides to cross (usually the category’s dominant one). A BUILD/MINE that faces the wrong or empty tile while still uncommitted is a harmless no-op and does not commit. A one-time *commit cost* is charged on the committing action, so when a short detour exists it can be better never to commit (leaving a small `none` mass). The commitment is exposed in the observation scalars ($\mathbb{R}^7 = [\text{FACING}(4), \text{step}/\text{max}, \text{committed_build}, \text{committed_mine}]$).

The belief→skill map. Crossing the *abundant* obstacle of a category is cheaper than detouring around all of it, so the better commitment is biased by the category: LAKES favour committing to *build*, ROCKY to *mine*, BALANCED either. Because commitment is implicit (the first obstacle the agent crosses), and on a one-sided map that first crossing is usually the dominant obstacle, the commitment naturally correlates with the category. We read this off as a 3×3 **commit matrix** (Fig. 2): rows are the map category, columns are the skill the agent ended the episode committed to (`none/build/mine`), and each entry is the fraction of held-out episodes of that category — so every row sums to 1. We are after a *soft* bias toward the matching tool (e.g. lakes leaning to build, rocky to mine) with a little `none` mass, not a hard diagonal — crossing is never strictly mandatory, since a detour usually exists.

Maps and reward. Terrain uses the *same* domain-warped fractal pipeline as the open task (§4.1), only with each category’s water/rock fractions; every map is rejection-sampled to be winnable under its intended commitment. The reward is unchanged (slack -0.01 , reach $+3$, no build cost, PBRS shaping with $\gamma=0.997$) with one addition: the cost-to-go potential is **commitment-aware**. Three Dijkstra fields (`none / build / mine`) are precomputed per map and indexed by the *live* commitment — the *non*-committed obstacle becomes an impassable wall — and capped at $2(H+W)$ so the potential stays bounded. Committing to the wrong tool therefore *raises* the cost-to-go (or strands the agent on a one-sided map), so the shaping itself, together with the sparse reach bonus, teaches the agent to read the terrain *before* it commits. The pure-JAX port is again proven bit-for-bit equivalent to the PyTorch env, now across all 8 actions.

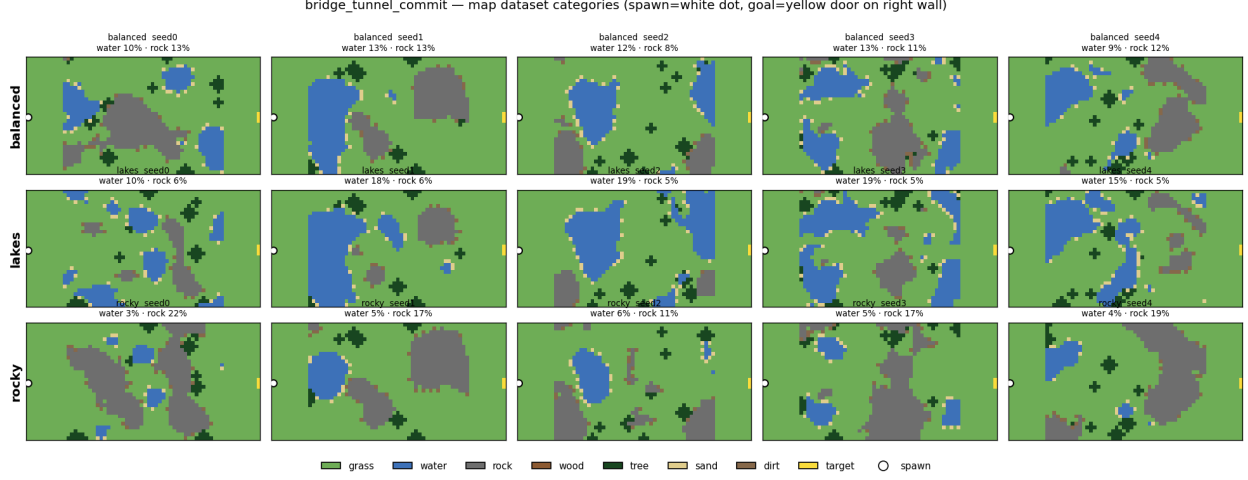


Figure 1: The three labeled map categories (rows) of `bridge_tunnel_commit`, five seeds each. White dot = spawn; yellow = the 3-cell goal door. LAKES are water-dominated (the cheap crossings are over water $\rightarrow$ commit *build*), ROCKY are rock-dominated (commit *mine*), BALANCED is an even mix (either tool works). The category is the ground-truth *belief* the agent must infer under partial observation.

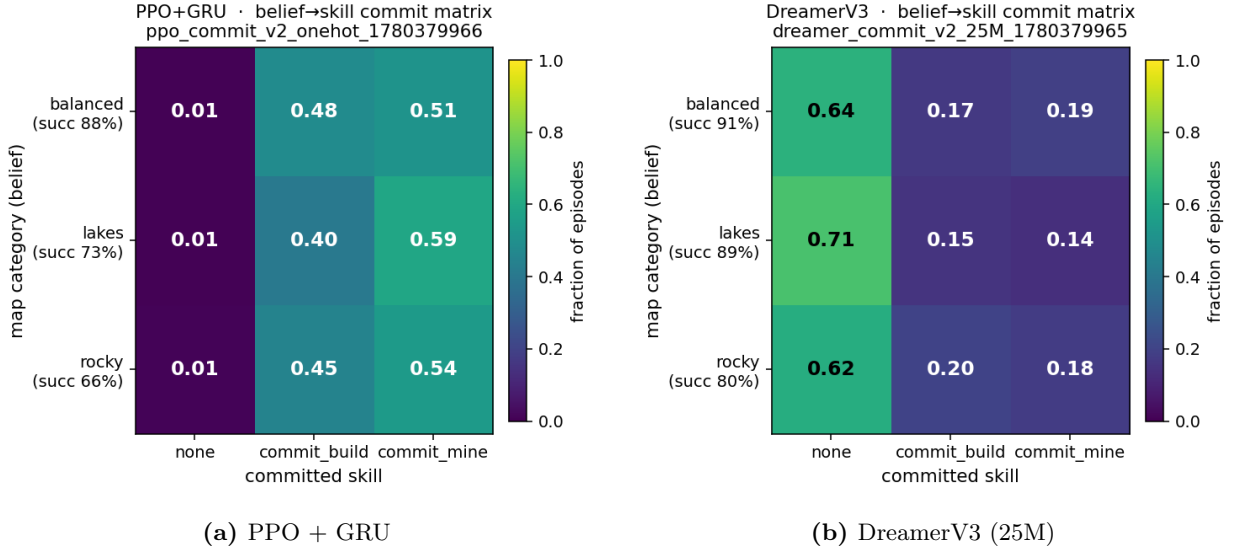


Figure 2: The 3×3 **belief $\rightarrow$ skill commit matrix** for each agent: rows = map category (the inferred belief), columns = the skill the agent irreversibly committed to (none/build/mine), entries = fraction of held-out episodes (each row sums to 1). Mass on the diagonal (build $\leftarrow$ lakes, mine $\leftarrow$ rocky) means the agent reads the terrain and commits correctly.

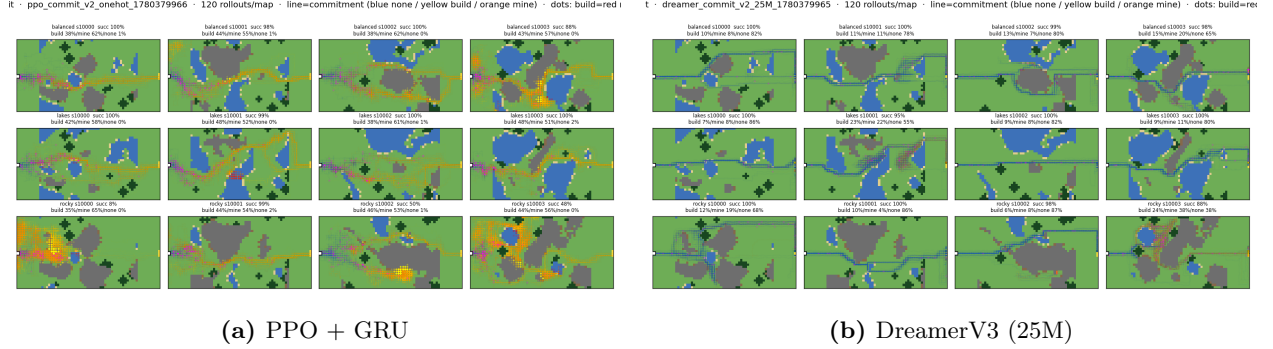


Figure 3: Stochastic rollouts on held-out maps, rows = category, columns = seeds. path = blue, bridge = red, mine = yellow, commit step = *. Each panel is titled with its success rate and the episode-level commit split.

From controlled to open. The remainder of the paper studies the original `bridge_tunnel` task, which removes *both* scaffolds. The belief is no longer a single labeled category: obstacle *type and size* vary continuously *within* a map and must be inferred afresh at each obstacle. And the skill is no longer enforced: there is no commitment, so the agent may bridge one obstacle, mine the next, detour a third, and even abandon a half-finished crossing. Belief and strategy survive only as *emergent, soft* variables in the recurrent state — which is the harder, more realistic setting the probing and steering programme of §9 targets.

3 Environment

Tiles. A 9-tile symbolic vocabulary (no decorative classes, so the decoder cannot interpolate phantom tiles):

id	tile	role	behaviour
0	GRASS	open ground	walkable (cost 1)
1	WATER	lake	cross by <i>bridging</i> (place → wood)
2	ROCK	mountain	cross by <i>mining</i> (mine → grass)
3	WOOD	placed bridge	walkable
4	TARGET	goal door	walkable; stepping on it wins
5	OOB	off-map pad	egocentric padding only
6	TREE	forest	impassable (detour only)
7	SAND	water fringe	cosmetic, walkable like grass
8	DIRT	rock fringe	cosmetic, walkable like grass

Observation (POMDP). An egocentric crop centred on the agent: $\text{minimap} \in \{0, \dots, 8\}^{V \times V}$ with $V = 21$ (off-map cells padded with OOB), plus a scalar vector $\text{scalars} \in \mathbb{R}^5 = [\text{FACING one-hot}(4), \text{step}/\text{max}]$. The agent never sees the whole map, so the size/type of an obstacle must be *inferred* as it approaches.

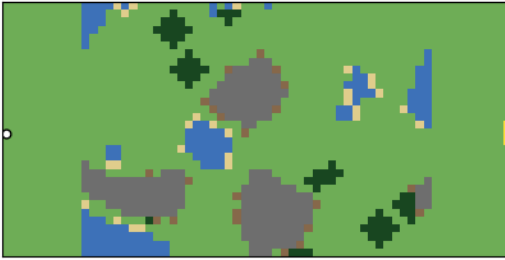
Actions. `Discrete(6)`: up / down / left / right (a move also sets FACING), `PLACE` (bridge the water cell in front: WATER → WOOD), and `MINE` (mine the rock cell in front: ROCK → GRASS). Trees can never be removed.

4 Task and procedural maps

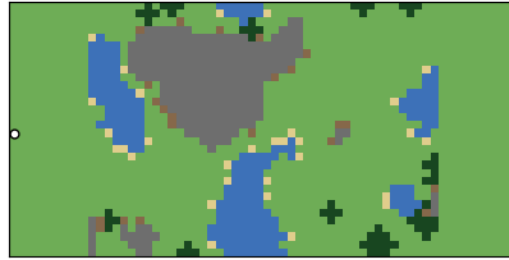
Maps are 32×64 , generated by a domain-warped fractal heightmap thresholded into lakes (**WATER**) and mountains (**ROCK**) with cosmetic **SAND/ DIRT** fringes; the left/right edge columns are kept obstacle-free. The **spawn** is the centre of the left edge and the **goal** is a 3-cell central door on the right wall (`goal_half = 1`). Crucially, impassable **tree forests are biased heavily toward the top and bottom rows** (about a 7:1 edge-to-middle ratio), which walls off the trivial “hug the wall to the door” route and forces traversal through the obstacle-filled centre. Figure 4 shows the curated validation set.

zebra_nav validation maps — natural 32x64

val map 0 (seed 10000)



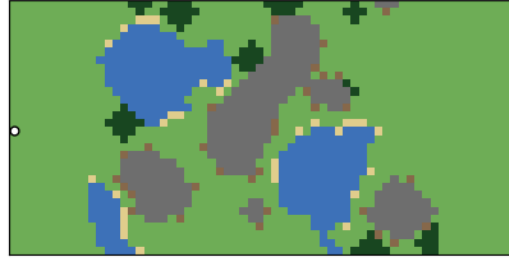
val map 1 (seed 10001)



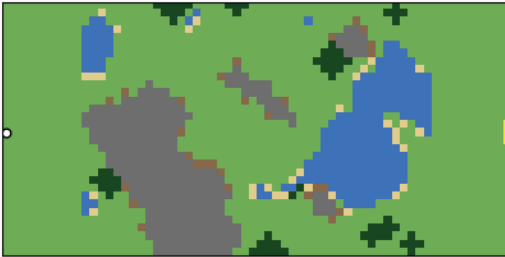
val map 2 (seed 10002)



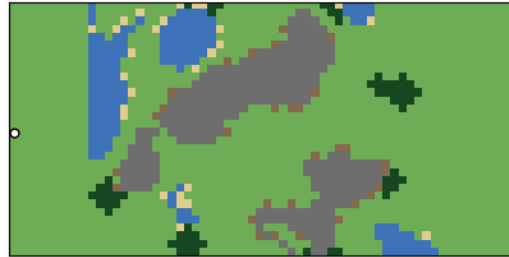
val map 3 (seed 10003)



val map 4 (seed 10004)



val map 5 (seed 10005)



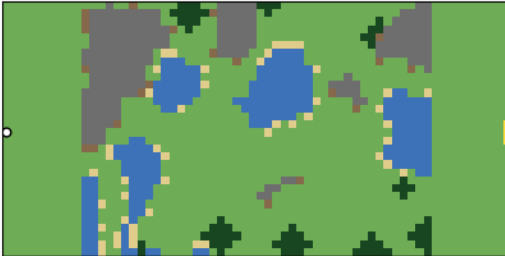
val map 6 (seed 10006)



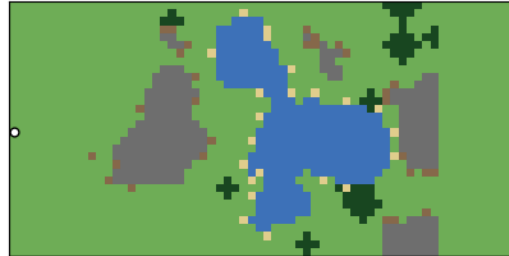
val map 7 (seed 10007)



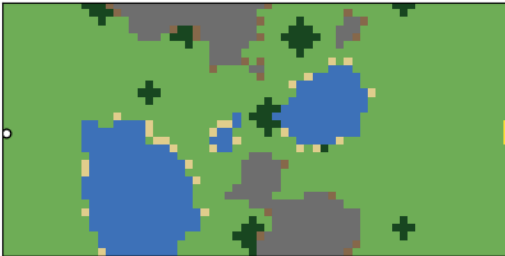
val map 8 (seed 10008)



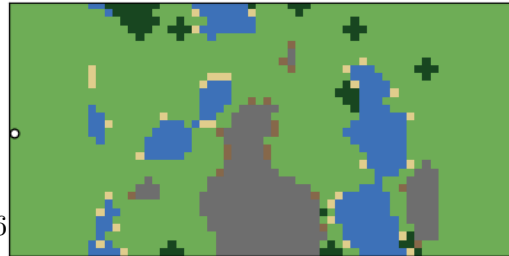
val map 9 (seed 10009)



val map 10 (seed 10010)



val map 11 (seed 10011)



val map 12 (seed 10012)



val map 13 (seed 10013)



4.1 Generation algorithm

Each map is produced deterministically from a single integer seed (`generate_bridge_tunnel_map(orientation="n", seed)`); the steps are:

1. **Domain-warped fractal heightmap.** A multi-octave OpenSimplex field h_0 (base frequency $\max(6, \max(H, W)/4.5)$, five octaves with weights $1, \frac{1}{2}, \frac{1}{4}, \frac{1}{8}, \frac{1}{16}$) is *domain-warped* by two further simplex fields (warp amplitude $0.10 \max(H, W)$): $h(r, c) = h_0(r + a w_r, c + a w_c)$. The warp removes grid-aligned, blobby structure and yields organic coastlines.
2. **Overlaid landforms.** 2–4 lakes (subtract a height bump) and 2–4 mountains (add one) are stamped, each a soft logistic *disk* or a multi-lobe *massif*; radii are drawn with a squared (small-biased) distribution so most features are small with a rare large one. A thin wiggly *ridge* (capsule field) is added with probability $\frac{1}{2}$.
3. **Quantile thresholding.** The lowest `water_frac` = 0.14 fraction of h becomes **WATER**, the highest `rock_frac` = 0.14 becomes **ROCK**, the rest **GRASS**. Thresholding by quantile makes obstacle coverage exact regardless of the noise realisation.
4. **Cosmetic fringes.** A sparse one-cell **SAND** speckle is dilated around water and **DIRT** around rock (each kept with prob. 0.30/0.25); both behave exactly like grass but visually mark shore-lines/scree.
5. **Edge bands & endpoints.** The left/right 10 columns are forced to **GRASS** (clean spawn/goal approaches). Spawn = $(H/2, 0)$; the goal is the 3-cell door $\{H/2-1, H/2, H/2+1\}$ on the right wall.
6. **Edge-biased forests.** About `tree_frac`· HW cells of impassable **TREE** are placed as radius-1–2 patches. Each patch row is sampled with a heavy edge bias — distance-from-nearest-wall $d = \frac{1}{2}u^3$ with $u \sim U(0, 1)$, snapped to the top or bottom edge — so most forest sits in the outer ~15% of rows, leaving the vertical middle open. A breadth-first **reachability guard** removes any patch that would wall the goal off from the spawn.
7. **Caching.** The dataset is generated once and pickled; the pure-JAX env additionally precomputes a Dijkstra cost-to-go field per map so its PBRs potential is a table lookup (and is identical to the PyTorch env by construction).

Why this geometry. An earlier variant used the *whole* right wall as the goal; with a free top/bottom border the cheapest route is to hug an edge and sprint right, and both agents collapsed to wall-hugging with no obstacle crossing. Making the goal a narrow central door *and* foresting the edges restores the intended behaviour: the agent must go through the middle and repeatedly choose bridge / mine / detour. The strategy mix is therefore a property of the map geometry, not of the exploration schedule.

5 Reward

A move-minimising shaped reward with no per-strategy bias:

$$r_t = \underbrace{-0.01}_{\text{slack}} + \underbrace{\text{shaping}_t}_{\phi = -\text{ctg}} + \underbrace{3.0 \cdot \mathbb{I}[\text{reach door}]}_{\text{terminal}}, \quad \text{shaping}_t = 0.015 (\text{ctg}_{t-1} - \gamma \text{ctg}_t),$$

with $\gamma = 0.997$. The potential $\phi = -\text{ctg}$ uses a **min-action cost-to-go**: entering WATER/ROCK costs 2 (build + move), walkable tiles cost 1, and TREE is impassable; **ctg** is computed once at reset by Dijkstra seeded from the goal cell. With per-build cost 0, crossing and detouring are valued purely by total action count, so which strategy the agent picks at a given obstacle is driven by geometry (is bridging/mining shorter than going around?), exactly the decision we want to study.

6 Agents

Both agents train on the *same* task to a shared metric schema. The PyTorch PPO env (`BridgeTunnelEnv`) and the pure-JAX env (`BridgeTunnelJaxEnv`) are proven bit-for-bit equivalent (a parity test asserts identical obs / reward / termination over thousands of steps), so the DreamerV3 world model trains on exactly the task the PPO agent does.

6.1 Recurrent PPO baseline

A tile-embedding CNN $\rightarrow$ GRU recurrent policy (Fig. 5). The $V \times V$ tile crop (9 classes) is embedded (16-d) and augmented with two CoordConv channels, passed through three 3×3 ReLU convolutions, flattened, concatenated with the 5 scalars, and projected to a 256-d embedding. A 128-unit GRU carries memory across the episode; **this $h_t \in \mathbb{R}^{128}$ is the POMDP belief carrier and the primary probing target**. Two linear heads read it: a categorical actor over the 6 actions and a scalar value. There is *no* auxiliary belief head — bridge/mine/detour is driven purely by reward. Entropy 0.045, no annealing (keeps the stochastic policy spread out). Adam ($\text{lr} = 3 \times 10^{-4}$), GAE ($\gamma=0.997, \lambda=0.95$), clip 0.2.

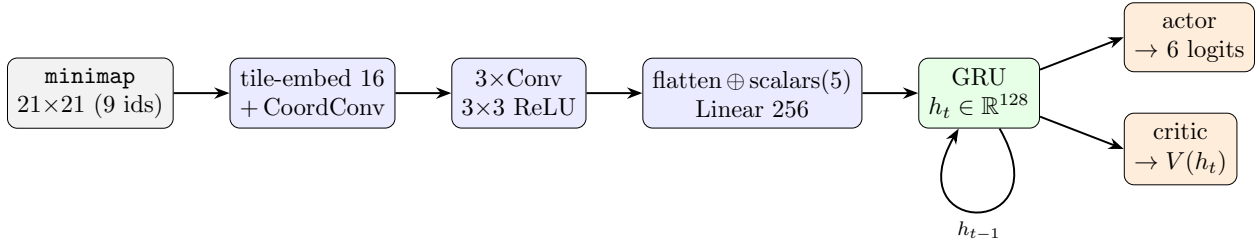


Figure 5: PPO+GRU policy. The 5 scalars are the FACING one-hot(4) plus the step fraction. The 6 actions are up/down/left/right/place/mine. The recurrent h_t is the only place the partially-observed obstacle ahead can be integrated, hence the probing target.

6.2 DreamerV3 world model (25M)

The in-tree pure-JAX `purejaxwm` library learns a recurrent state-space model (RSSM) and trains the actor–critic purely *in imagination* (Fig. 6). Because the minimap is symbolic, the encoder/decoder are 4-block MLPs (RMSNorm + SiLU) over the flat $V \cdot V + 5$ observation vector — no pixel CNN. Heads read the RSSM features $[h_t, z_t]$: a decoder $\hat{o}_t$, TwoHot reward/continue heads, and the actor/critic optimised on imagined λ -returns with a slow (EMA) critic.

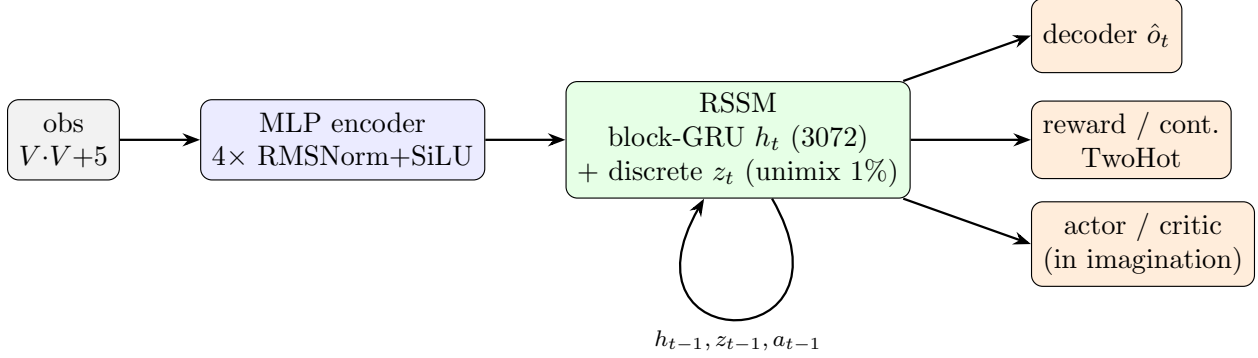


Figure 6: High-level DreamerV3. The RSSM rolls forward in latent space; the actor-critic is trained on imagined trajectories ($\gamma=0.997, \lambda=0.95$, TwoHot over 255 symlog bins, RetNorm scaling, LaProp + AGC, $\text{lr} = 4\text{e-}5$). The h_t (3072-d for the 25M preset) + discrete z_t are the latent we probe.

6.3 A fair comparison

To compare the two algorithms cleanly we train them on the *identical* environment *and* the *identical observation*: both consume a **categorical one-hot** minimap ($V \times V \times 9$). For PPO this replaces the learned tile embedding with one-hot channels into the CoordConv CNN; for DreamerV3 it replaces the ordinal-scalar encoder/decoder with a one-hot encoder and a per-cell cross-entropy decoder (§8). Only the *algorithm* then differs. Table 1 reports held-out trajectory-grid success (200 stochastic rollouts $\times$ 6 maps); Figure 7 shows the training curves.

agent	observation	success (held-out)	return	episode length
PPO + GRU	one-hot	100%	≈ 3.4	≈ 89
DreamerV3 (25M)	one-hot	85%	≈ 2.1	longer / variable

Table 1: Both on the same env + same categorical observation (only the algorithm differs). DreamerV3 logs $\approx 96\%$ success *during* training but $\approx 85\%$ in clean per-episode evaluation at 1M steps (it was only $\approx 7\%$ at 500k — door-homing on the narrow 3-cell goal is what the extra budget buys).

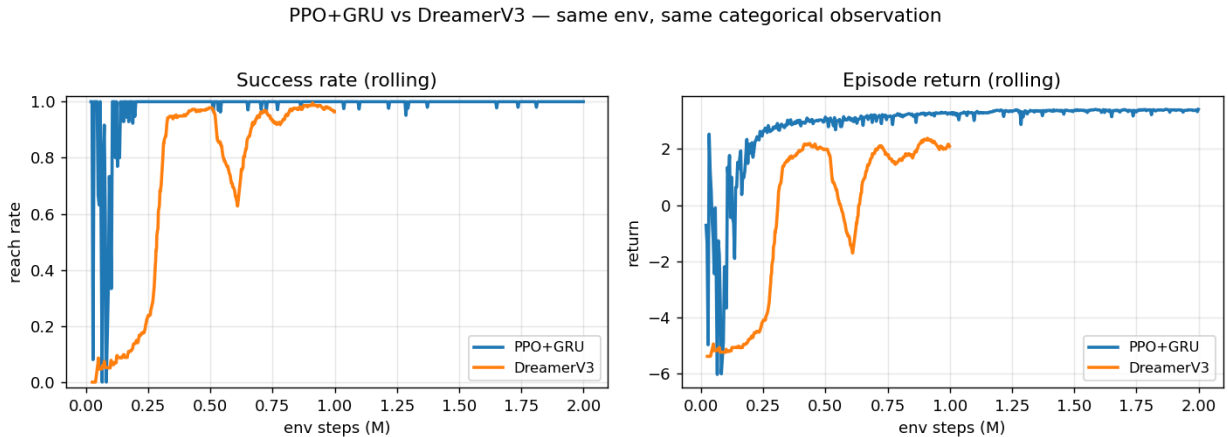


Figure 7: Reward and success vs. environment steps (W&B, rolling). On this symbolic task **PPO+GRU is markedly more sample-efficient** — $\sim 100\%$ by $\sim 0.2\text{M}$ steps — while DreamerV3 rises more slowly (with a mid-training dip near 0.5M) to $\sim 96\%$ by 1M. PPO trained to 2M, DreamerV3 to 1M.

7 Behaviour: stochastic rollouts

Figure 8 overlays 200 stochastic rollouts per map for each agent. The forests pin the paths into a central corridor; within it the agents bridge (red), mine (yellow), and detour around the larger masses. Figure 9 isolates one clean rollout of each strategy from the PPO agent — the three behaviours we want to find and manipulate internally.

What counts as a crossing (strict). We label a trajectory segment as *bridging* or *tunnelling* only when the path **enters an obstacle body and exits the other side**: a maximal run of consecutive path cells whose original terrain is WATER (resp. ROCK), with at least 2 distinct cells *and* distinct entry/exit land cells. Placing or mining a single block, or poking into a lake and backing out the same side, does *not* count — it is still “avoid”. This is deliberately stricter than counting build/mine events, and it is exactly the segment-labelling rule we will use for the strategy labels in the activation dataset (§9).

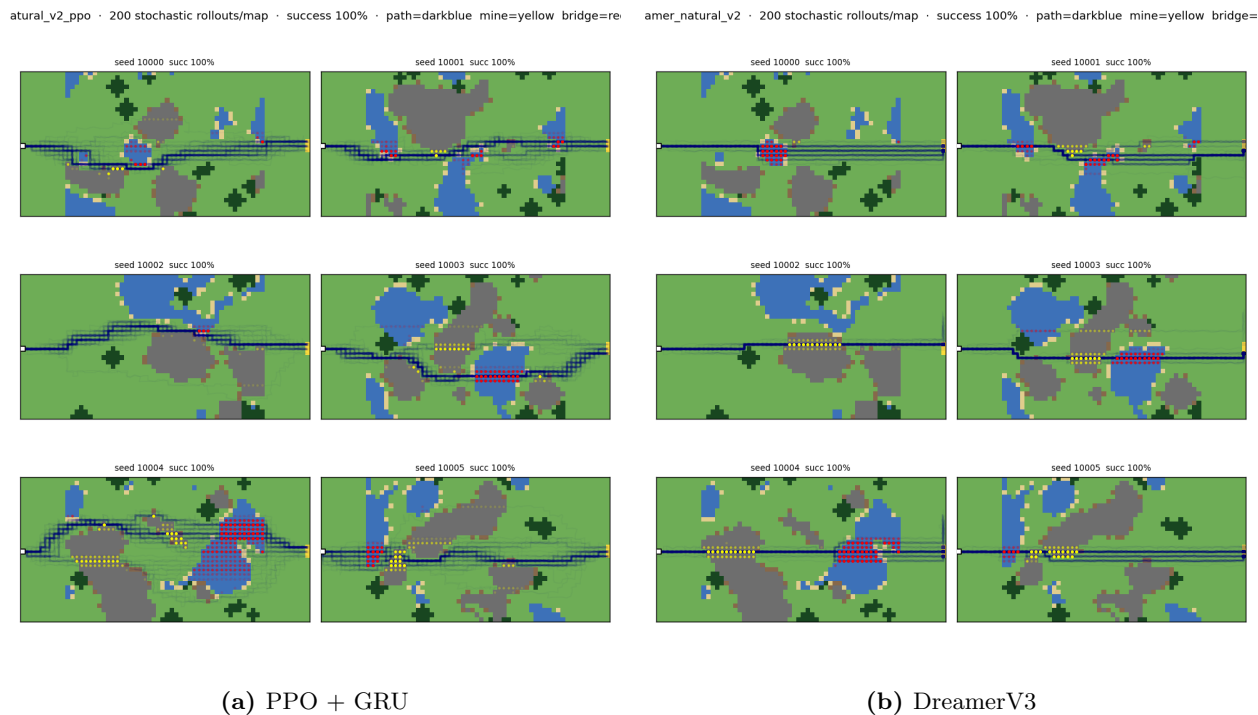


Figure 8: 200 stochastic rollouts per map. path = dark blue, bridge = red, mine = yellow. Both agents funnel through the forest-walled central corridor and cross obstacles there.

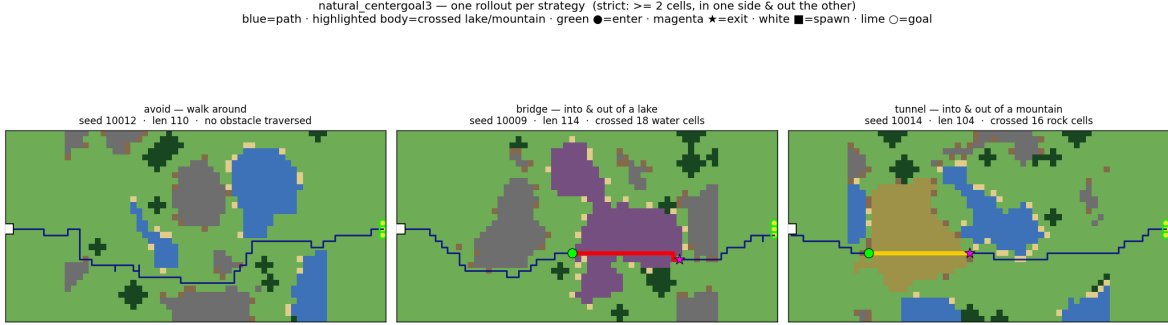


Figure 9: One stochastic rollout per strategy (PPO agent), under the strict crossing definition. The crossed lake/mountain body is shaded; the bold segment is the in→out traversal (green ● = entered the body, magenta ★ = exited the far side), the thin blue line is the rest of the path. **Avoid** traverses no obstacle body; **Bridge** crosses a multi-cell lake (18 water cells here); **Tunnel** crosses a mountain (16 rock cells). Over sampled reaching rollouts the mix is roughly avoid:bridge:tunnel $\approx 1:2:0.8$, so all three modes are well represented.

8 DreamerV3 imagination

Because DreamerV3 learns a world model, we can roll it forward *in imagination*: condition the RSSM on a few real frames, then let it predict the rest open-loop (prior only, no further observations) while the chosen actions still drive the real env, and decode each latent back to an egocentric view. The minimap is decoded with a **per-cell categorical (cross-entropy) head** over the 9 tiles (the categorical observation of §6.3). Figure 10 compares the model’s decode (top) against the **ground-truth** env observation (bottom) on a demo map.

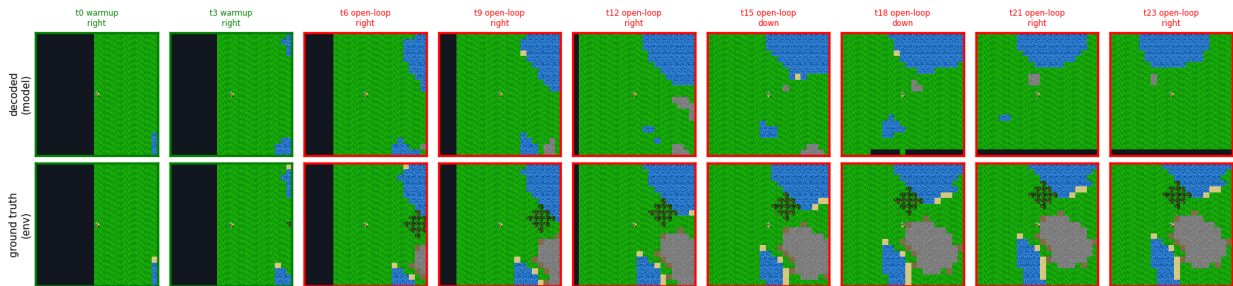


Figure 10: Open-loop prediction vs. reality on a demo map (sprites). **Top row** = the model’s decoded egocentric view; **bottom row** = the ground-truth env observation at the same step. **Green** columns are the closed-loop warmup (posterior conditioned on the real obs, so the top row is a faithful *reconstruction*); **red** columns are open-loop imagination (prior only). The reconstruction tracks reality closely; the open-loop prediction stays plausible (grass/water/rock) but drifts from the true map — it does not reproduce the tree cluster and occasionally hallucinates off-map (black) cells. (Note: the top warmup frames are decoder reconstructions, *not* the raw observation — which is exactly the bottom row.)

Why categorical (relevant to probing). An earlier version encoded the minimap as a single *ordinal scalar* per cell (tile_id/NUM_TILES) and *regressed* it with MSE. That produced blurry reconstructions and *phantom out-of-vocab tiles*: a value between, say, rock(2) and wood(3) rounds to an id that never occurs. The categorical one-hot encoder + per-cell cross-entropy decoder

removes both pathologies — reconstructions are crisp and can only emit real tiles (above) — at a modest cost in policy sample-efficiency (Fig. 7). The residual divergence in the open-loop frames is therefore *dynamics* error (the RSSM prediction), not a decoding artifact. For the interpretability work this matters: the decode is now a genuine per-cell tile belief, but we will still probe the latent (h, z) directly rather than read beliefs off the decoder.

9 Mechanistic-interpretability plan

The agents above are the substrate. The intended programme:

1. **Activation dataset.** Fix one or two maps, sample many stochastic trajectories (logging seed + full metadata), and record the GRU state h_t (and the pre-recurrent encoder embedding) at every step, labelled with the ground-truth belief (obstacle type/size ahead) and the committed strategy (avoid / bridge / tunnel) at each obstacle.
2. **Probe.** Fit linear probes / difference-of-means directions for the belief and strategy variables; report decoding accuracy by site, and validate with unsupervised structure (PCA / clustering).
3. **Steer.** Add / clamp those directions during fresh rollouts and measure the causal behavioural change (a probe that decodes but does not steer is suspect).
4. **Suppress.** Derive the “mine” direction from the *decision* contrast (mined vs. went-around on the same rock obstacle), apply it conditionally, and verify the agent re-routes via its own bridge/detour policy rather than failing — i.e. a competent agent that **never digs tunnels**, achieved in activation space.

Reproduce. Train: `scripts/train_ppo_bridge_tunnel.py --config models/bridge_tunnel_nav/natural_` and `scripts/dreamerv3_bridge_tunnel_nav.py --size 25M`. Visualise: `scripts/bridge_tunnel_traj_grid.py`, `scripts/bridge_tunnel_strategy_examples.py`, `scripts/viz_dreamer_bridge_tunnel_imagine.py`. See `docs/bridge_tunnel_nav.md` and `docs/codebase_map.md`.